# Design the GET Endpoint for Listing Products

Explore how to design the GET endpoint for listing products in a REST API. Understand the principles of the GET method including idempotency, caching, request headers, response headers, and how to handle data formats. Gain hands-on knowledge of structuring HTTP requests and responses to comply with standards while using Python and Flask.

One of the main operations of any API is to provide a way for the client to fetch all the available collections related to a resource. In our case, the resource is the product and the collection is the list of all products with their details. In HTTP, this is done via the GET method.

## Listing products#

Before getting into the details of the endpoint, we need to keep the following details of the GET method in mind so that the API follows the HTTP specification:

- GET can only be used to retrieve data.
- A GET request has to be idempotent, i.e., the effect of multiple GET requests should be the same as a single GET request. For example, multiple GET calls with the same product ID should retrieve the same product.
- GET shouldn't modify the resource associated with the request, i.e., a GET request shouldn't make a change to the resource as a result of the ca
- A GET request can be cached either on the server side or the client side since the GET request identifies a resource.

Ask

- A GET request can be bookmarked by the client since it's associated with a URI. Hence, it's recommended that API shouldn't expose the GET endpoints whose URLs can change dynamically.
- Only text (ASCII or Unicode) data can be sent as request data.
- API should consider the limit of data the server can accept. All the data in the GET request is sent as a part of the URI itself, as there is no body associated with GET. This limits the amount of data that can be sent as a part of the GET request from the client.
- The response provided by GET should be in the format specified in the request. If this isn't possible, the API should send back an error.

### A note on idempotency

As per the specification:

- A request method is considered idempotent if the intended effect on the server of multiple identical requests with that method is the same as the effect for a single such request. Of the request methods defined by this specification, PUT, DELETE, and safe request methods are idempotent.
- Request methods are considered safe if their defined semantics are essentially read-only, i.e., the client doesn't request and expect any state change on the origin server as a result of applying a safe method to a target resource. Of the request methods defined by this specification, the GET, HEAD, OPTIONS, and TRACE methods are defined to be safe.

## Request/response structure

Ask

Based on the above points, let's look at the structure of the GET request and response.

### Request header

The request header details what the client wants and what the client will accept as a response. Following are some request headers that can be used in a GET request:

- **Method:** This is the HTTP method to be executed by the server. The method will be GET in this case.
- **URL/URI:** This is the identifying URI of the resource.
- **Version:** This is the version of the HTTP, i.e., 1.0, 1.1, and so on.
- **Accept:** This is the MIME type/format of the body of the response that the client will accept. It's very important in GET requests, as based on the Accept header, the server decides the format of the data. Generally, there's always a default format. However, that's not guaranteed.
- **User-Agent:** This is set by the client that sends the request and is optional.

Apart from these, there are headers that are specific to GET. The most commonly used among them are:

- **If-Modified-Since:** It's used to tell the server to return the entity if the entity has been modified since the date and time supplied with the header. For example, `If-Modified-Since: Sat, Jan 29, 2022, 19:43:31 GMT` tells the server to return the requested entity only if it has been updated after 7:43 PM of 01-29-2022.
- **If-Unmodified-Since:** It's the opposite of If-Modified-Since. In this case, it's used to tell the server if the entity hasn't been modified since the supplied date and time. For example, `If-Unmodified-Since: Sat, Jan 29, 2022, 19:43:31 GMT` tells the server to return the requested entity only if it hasn't been updated after 7:43 PM of 01-29-2022.

> The above-mentioned headers are used to make a GET request conditi
> in case caching is used.

Ask

### Response header

The response header provides the details sent by the server after processing the request. The most commonly used headers are:

- **Cache-Control:** This tells the client whether the response can be cached or not. If yes, then it defines parameters such as interval, and URL of caching server. For the GET response, this header is crucial as the response is cacheable by default. If the response can't be cached or if the cache's life is low, then the parameters must be set as a part of this header.
- **Content-Type:** This is the format or MIME type of the content in the body. This should be the same as that of the Accept header of the request. If this isn't possible, the value of the Status header should be set to `406 NOT ACCEPTABLE`.
- **Content-Length:** This is the size of the content within the body.
- **Status:** This is the status of the request. If the request has been processed successfully, the status will be `200 OK`. If unsuccessful, it can be any of the error codes based on the reason for the failure.

### Response body

The body will contain the response entity. In our case, it'll be a list of products. The format of the response entity (specified by the Content-Type of the response header) should be the same as that specified by the Accept header.

Ask

Request/response structure for the GET API for a list of products

## Products API

Keeping the above points in mind, the request headers that will be used are as follows:

index.js Javascript (babel-node)

```
GET /api/products HTTP/1.1
Host: localhost:5000
User-Agent: curl/7.71.1
Accept: application/json
```

Ask

The `User-Agent` header will be optional.

The response header provided by the API will be:

index.js Javascript (babel-node)

```
HTTP/1.1 200 OK
Content-Type:application/json
Content-Length:100
```

The response body sent by the server will be:

index.js Javascript (babel-node)

```
[
  {
        "name": "servlet-container",
        "description": "container for servlet",
        "id": "1",
        "version": "3.0"
  },
  {
        "name": "ejb-container",
        "description": "container for ejb",
        "id": "2",
        "version": "3.0"
  }
```

Ask

**Note:** The list of products given above is just for the sake of example.